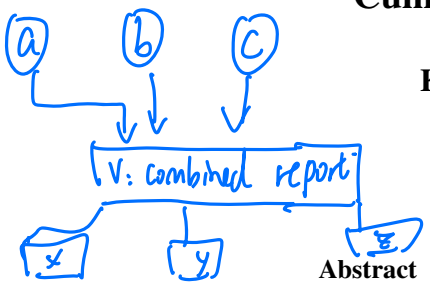
arXiv:2608.14668v1 [cs.MA] 2 Aug 2026

BRA-Audit: Budgeted Runtime Auditing for LLM Multi-Agent Systems via Cumulative-Exposure Audit-Point Placement

Kaixiang Wang, Yidan Lin, Jiong Lou, Jie Li



Abstract

LLM-based multi-agent systems (LLM-MAS) solve complex tasks through specialized collaboration, but inter-agent dependencies can propagate hallucinated or malicious outputs into system-level failures. Auditor agents mitigate these risks, yet existing strategies face an efficiency dilemma: end-only auditing reviews long trajectories and final outputs, potentially weakening audit effectiveness and enlarging rollback scope, while auditing every agent each round improves detection and localization at high token cost. How can guard performance be preserved while minimizing token cost? To address this problem, we propose BRA-Audit, a budget-aware runtime auditing framework that models MAS execution as a dynamic dependency graph and formulates audit scheduling as audit-point placement under a fixed audit-call budget to minimize cumulative unchecked exposure. Its greedy scheduler prioritizes influential and long-unaudited regions, while trusted audit points enable localized recovery. Across structured coordination, complex reasoning, and open-ended tasks, BRA-Audit restores performance close to the clean setting, remains competitive with heavy guard methods and reduces end-to-end token consumption by 17.2%–40.6%.

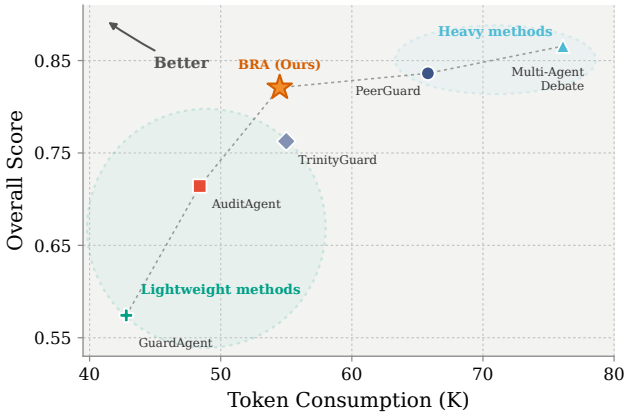


Figure 1: The performance-cost trade-off of BRA-Audit against other methods under attack in AgentsNet benchmark.

Introduction

LLM-based multi-agent systems (LLM-MAS) coordinate specialized agents through explicit roles, communication, and shared workflows (Hong et al. 2024; Qian et al. 2024; Chan et al. 2024). Recent systems demonstrate that MAS can support generalist task execution, diverse coordination protocols, multi-agent reflection, and layered response aggregation (Fourney et al. 2024; Zhu et al. 2025; Wang et al. 2024). Other studies optimize communication graphs and architecture selection to remove redundant exchanges and adapt computation to task difficulty (Zhang et al. 2024; Li et al. 2024; Guo et al. 2024). These advances suggest that LLM-MAS can combine task decomposition, heterogeneous expertise, tool use, and multiple reasoning paths, making them promising for long-horizon and complex open-ended tasks (Tran et al. 2025; Yan et al. 2026; Li et al. 2023).

However, multi-agent coordination also expands the failure surface. Hallucinated or malicious messages can be reused as context, alter downstream decisions, and propagate across agents (Lee and Tiwari 2024; Ju et al. 2024; He et al. 2025). Recent attacks can manipulate exchanged messages, hijack execution control, leak system information, or conceal malicious intent (Triedman, Jha, and Shmatikov 2025; Wang et al. 2025a; Xie et al. 2025). Because agents repeatedly consume one another's outputs, a local error may evolve into a system-level failure, while its origin becomes difficult to identify in long and tightly coupled trajectories (Xia et al. 2026; Zheng et al. 2025).

Past safeguards detect unreliable behavior through self-reflection or graph-based anomaly detection. However, self-reflection may reproduce the generator's failure modes, while GNN-based detectors often rely on labeled attacks and generalize poorly (Shinn et al. 2023; Renze and Guven 2024; Wang et al. 2025b; Miao et al. 2026). Dedicated auditor agents have therefore become a common alternative, providing stronger and more transferable oversight through runtime inspection and cross-agent reasoning (Xiang et al. 2024; Wang et al. 2026; Fan and Li 2025; Du et al. 2024). Nevertheless, existing studies mainly improve *how* auditing is performed, with limited attention to *when* and *where* auditors should be invoked. End-only auditing examines the full execution trajectory and final result after task completion. Its long, dependency-dense context can obscure failure sources, weaken audit effectiveness, delay intervention, and increase rollback costs (Liu

et al. 2024; Yang et al. 2026). In contrast, interaction-level auditing, as exemplified by PeerGuard (Fan and Li 2025), inspects agents at each round and enables earlier detection and finer localization. However, repeated auditor calls incur substantial token costs and waste computation on benign interactions (Du et al. 2024). This raises a significant question: *how can we determine when and where to audit while preserving guard performance and minimizing token cost?*

To answer this question, we propose BRA-Audit, a budget-aware runtime framework that directs limited audit resources to critical points in LLM-MAS execution. BRA models inter-agent communication and execution dependencies as a dynamic graph, where nodes represent runtime outputs and edges capture information flow. It then formulates audit scheduling as an audit-point placement problem that minimizes cumulative unchecked exposure under a fixed budget. The scheduler combines audit gaps with topological influence, prioritizing regions that have remained unchecked and can strongly affect downstream computation. Each verified audit point becomes a trusted audit point in this round; when unsafe behavior is detected, BRA contains the affected region and rolls back only the necessary subtrajectory. By integrating topology-aware scheduling, bounded auditing, and localized recovery, BRA maintains effective runtime oversight while reducing redundant checks and re-execution. It therefore achieves an empirical trade-off between audit performance and token consumption.

Further experimental results show that: across MAS collaboration environments and complex open-ended tasks, BRA-Audit restores task performance close to clean baselines while remaining competitive with stronger auditing baselines. It reduces token consumption by 17.2%–40.6% compared with heavy guard like PeerGuard and Multi-Agent Debate, demonstrating a favorable trade-off between defense effectiveness and token cost (as shown in Fig. 1).

Our contributions are as follows:

- To determine when and where to audit an LLM-MAS while minimizing cost without compromising audit effectiveness, we formulate budget-constrained auditing as an audit-point placement problem over dynamic execution graphs.
- We propose BRA-Audit, an online budgeted audit-point placement framework in LLM-MAS that sequentially selects audit points by marginal cumulative-exposure reduction. It combines audit gaps and downstream dependency reachability to weight unchecked exposure, while verified audit points bound audit contexts and localize rollback.
- We conduct extensive experiments in MAS collaboration environments and complex open-ended tasks. BRA-Audit achieves competitive recovery performance while reducing token consumption by 17.2%–40.6% compared with strong auditing baselines, demonstrating a favorable empirical Pareto trade-off.

Related Work

Attacks and Failures in LLM-MAS. LLM-based multi-agent systems are vulnerable because agents exchange and reuse intermediate outputs, allowing local errors to escalate into system-level failures. Systematic analysis identifies failures in system design, inter-agent alignment, and task verification, revealing collaboration-specific risks beyond isolated agents (Cemri et al. 2025). Adversaries can exploit the same dependencies: malicious instructions can self-propagate across agents (Lee and Tiwari 2024), manipulated knowledge can spread through conversations and persist in shared retrieval memories (Ju et al. 2024), and communication attacks can intercept or modify inter-agent messages without compromising individual agents (He et al. 2025). More severe control-flow hijacking can invoke unsafe tools, exfiltrate sensitive data, or execute malicious code (Triedman, Jha, and Shmatikov 2025). Overall, communication, shared memory, and orchestration amplify benign errors and adversarial inputs, making early detection and containment essential for reliable LLM-MAS.

LLM Multi-Agent Safeguarding. Early guards use prompt-based self-reflection, but the reviewer may share the generator's failure modes (Shinn et al. 2023; Renze and Guven 2024). G-Safeguard applies a graph neural network to communication graphs to detect anomalous agents and prune risky links, yet relies on labeled attacks and may generalize poorly to unseen threats (Wang et al. 2025b; Miao et al. 2026). More reliable approaches introduce LLM auditors. GuardAgent converts safety requirements into executable checks, TrinityGuard coordinates runtime monitors, PeerGuard uses mutual reasoning, and multi-agent debate performs iterative cross-examination (Xiang et al. 2024; Wang et al. 2026; Fan and Li 2025; Du et al. 2024). These methods improve oversight, but their effectiveness and cost vary on complex tasks.

These safeguards methods either inspect trajectories too coarsely, which weakens detection and enlarges the rollback scope, or audit every interaction, which incurs substantial token and latency costs. We therefore propose BRA-Audit, a budgeted audit-point placement framework that preserves effective trajectory auditing while reducing redundant checks.

System Model

This section defines the LLM-MAS execution model and the threat model considered in this work.

MAS System

We consider a graph-structured LLM multi-agent system (LLM-MAS) with a set of agents

$$\mathcal{A} = \{a_1, \ldots, a_n\}. \tag{1}$$

Each agent has a predefined role and contributes to the task through its own interaction pattern, information sources, and tools. We represent runtime execution as a dynamic dependency graph:

$$G_t = (V_t, E_t). \tag{2}$$

Each node $v \in V_t$ represents a runtime object, such as an agent message, intermediate result, tool interaction, retrieved evidence, or aggregation result. It is associated with its producing agent $a(v)$, timestamp $t(v)$, and content $m(v)$. A

$G_t = (V_t, E_t)$

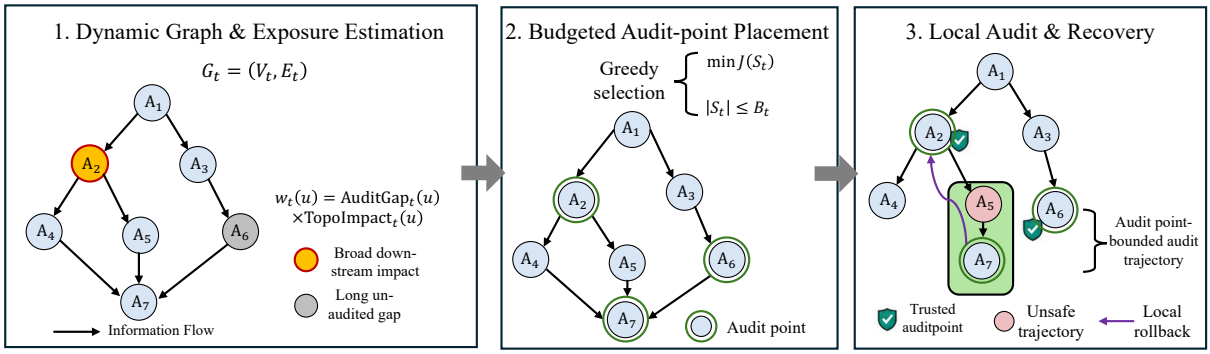


Figure 2: Overview of BRA-audit.

directed edge $(u, v) \in E_t$ indicates that v depends on u through communication, evidence usage, tool execution, or aggregation. A path in G_t therefore captures the propagation of information and potential errors.

Auditing is modeled as audit-point placement on the execution graph. An audit point is a selected node $s \in V_t$ inspected by an auditor using the relevant context between the previous audit point and s. If the trajectory is safe, s becomes a trusted audit point in this round. Otherwise, the system may block, sanitize, isolate, or roll back the affected trajectory.

Threat Model

Unsafe behavior may arise from compromised agents, hallucinations, tool errors, poisoned knowledge bases, corrupted websites, or misleading retrieved documents. Despite their different causes, these failures introduce false, unsafe, or misleading information into the execution graph.

We use *malicious agents* as a unified abstraction for these cases. Let

$$\mathcal{M} \subseteq \mathcal{A}, \tag{3}$$

denote the set of malicious agents. For a node $v \in V_t$, its content may be unsafe if it is produced by a malicious agent:

$$a(v) \in \mathcal{M} \Longrightarrow \Pr(y(v) = 1) > 0. \tag{4}$$

Here, $y(v) = 1$ denotes unsafe content. The term *malicious* does not require intentional behavior; it only indicates that an agent produces corrupted information that may mislead downstream agents.

A malicious agent may fabricate evidence, omit constraints, produce incorrect intermediate results, distort aggregation, or trigger unsafe tool usage. These outputs can propagate through dependency edges and affect later reasoning or the final result. We therefore study how runtime auditing detects and contains such propagated failures over the execution graph.

Method

This section formulates budgeted runtime auditing as an optimization problem and develops the mathematical model and scheduling algorithm used by BRA-Audit (Fig. 2 shows overview of BRA-audit).

Problem Formulation and Optimization Objective

LLM-based auditors can inspect intermediate MAS outputs and identify hallucinated, inconsistent, or unsafe reasoning steps (Zheng et al. 2023; Gu et al. 2025). As discussed in the introduction, final-only auditing requires processing a long execution trajectory and may cause large rollback, whereas auditing every interaction requires frequent auditor calls. We therefore study how to allocate a limited audit budget during MAS execution.

A successful audit verifies the audit point-bounded dependency region reaching the inspected node and establishes the node as a trusted audit point in this round for subsequent execution. The size of this region determines both the context inspected by the auditor and the execution scope that may need to be rolled back. Audit-point placement is therefore important: leaving a node with broad downstream reachability unchecked can repeatedly carry its exposure into later audit contexts, while a long audit gap indicates stale trust evidence for the corresponding agent or region. Moreover, audit point decisions are coupled because each new audit point reshapes downstream audit regions and changes the value of subsequent audit points.

Given a candidate set $Q_t \subseteq V_t$ and an audit-call budget B_t, BRA-Audit selects before execution an audit point set

$$S_t \subseteq Q_t, \qquad |S_t| \le B_t. \tag{5}$$

Each selected audit point triggers one auditor call over its audit-point bounded dependency region. The scheduler therefore seeks placements that keep audit regions compact while prioritizing influential nodes and agents that have remained unaudited for many rounds.

We formulate audit point placement as minimizing cumulative unchecked exposure:

$$\min_{S_t \subseteq Q_t} J(S_t) \quad \text{s.t.} \quad |S_t| \le B_t, \tag{6}$$

where $J(S_t)$ measures the total cumulative exposure remaining under audit-point placement S_t. By preventing unchecked information from being repeatedly inherited by downstream contexts, this objective jointly reduces audit context size and delayed-detection exposure. Here, B_t controls the number of auditor calls, while total token cost is evaluated end to end. We define $J(S_t)$ and its components in the next subsection.

why strict descendants

B_t: number of auditor calls

Is B_t the right metric

Algorithm 1: BRA: Exposure audit-point Scheduling

Require: Execution graph $G_t = (V_t, E_t)$, candidate set $Q_{t+1} \subseteq V_t$, audit ratio ρ, and last audit records $\mathrm{LastAudit}_t(\cdot)$

Ensure: Selected audit points S_{t+1} and updated audit records $\mathrm{LastAudit}_{t+1}(\cdot)$

$B_{t+1} \leftarrow \lfloor \rho |Q_{t+1}| \rfloor$
$S_{t+1} \leftarrow \emptyset$
for each node $u \in V_t$ **do**
  $\mathrm{AuditGap}_t(u) \leftarrow t - \mathrm{LastAudit}_t(a(u)) + 1$
  $\mathrm{TopoImpact}_t(u) \leftarrow 1 + |\mathrm{Desc}_t(u)|$
  $w_t(u) \leftarrow \mathrm{AuditGap}_t(u) \cdot \mathrm{TopoImpact}_t(u)$
end for
while $|S_{t+1}| < B_{t+1}$ and $Q_{t+1} \setminus S_{t+1} \neq \emptyset$ **do**
  for each $v \in Q_{t+1} \setminus S_{t+1}$ **do**
    $\Delta(v \mid S_{t+1}) \leftarrow J(S_{t+1}) - J(S_{t+1} \cup \{v\})$
  end for
  $v^\star \leftarrow \arg\max_{v \in Q_{t+1} \setminus S_{t+1}} \Delta(v \mid S_{t+1})$
  $S_{t+1} \leftarrow S_{t+1} \cup \{v^\star\}$
end while
deploy S_{t+1} before round-$(t+1)$ execution
for each audit point $s \in S_{t+1}$ in causal order **do**
  $\mathcal{T}(s) \leftarrow \mathrm{Region}(s \mid S_{t+1})$
  $r \leftarrow \mathrm{Auditor}(\mathcal{T}(s))$
  if $r = \mathrm{safe}$ **then**
    mark s as a trusted audit point for downstream execution in this round
  else
    block the affected downstream information flow
    rollback and recover the work products affected by $\mathcal{T}(s)$
    re-execute and re-audit $\mathcal{T}(s)$ before continuing
  end if
  update $\mathrm{LastAudit}_{t+1}(a(s))$
end for
return $S_{t+1}, \mathrm{LastAudit}_{t+1}(\cdot)$

Analysis and Solution

We now analyze the cumulative-exposure objective and derive the audit-point scheduling rule used by BRA-Audit. Algorithm 1 summarizes the main procedure of BRA-Audit. The key observation is that an unchecked node does not only affect its own audit context. Its exposure can continue to appear in downstream dependency contexts until the corresponding information flow reaches an audit point. Audit scheduling therefore considers how an audit point jointly reduces exposure in multiple downstream contexts.

Since runtime objects are ordered by their generation times and dependency edges follow causal information flow, repeated agent interactions are unrolled across rounds, and the runtime dependency graph G_t is represented as a directed acyclic graph.

For each node $u \in V_t$, let $a(u)$ denote its producing agent, and let $\tau_t(a)$ be the latest completed audit round of agent a available at scheduling time t. We define

$$\begin{aligned} \mathrm{AuditGap}(u) &= (t+1) - \tau_t(a(u)), \\ \mathrm{TopoImpact}_t(u) &= |\{u\} \cup \mathrm{Desc}_t(u)|, \\ w_{t+1|t}(u) &= \mathrm{AuditGap}(u)\, \mathrm{TopoImpact}_t(u), \end{aligned} \tag{7}$$

where $\mathrm{Desc}_t(u)$ denotes the strict descendants of u. We consider verification age because risk can accumulate while an agent's outputs remain unchecked, and downstream reach because errors at influential nodes can affect more subsequent computations and nodes with broad downstream reachability also tend to play more critical roles in the task. Including u itself ensures that even sink nodes retain nonzero local impact. Accordingly, the first term measures the verification age when the scheduled audit is executed, while the second measures the closed downstream region potentially affected by u. Their product serves as a temporal–topological exposure measure, prioritizing nodes with stale verification and broad downstream reach using runtime audit records and graph structure.

Let $\mathrm{Anc}_t(v)$ denote the ancestors of v. We write $u \rightsquigarrow_{S_t} v$ if there exists a directed path $p : u \rightsquigarrow v$ such that $(V(p) \setminus \{v\}) \cap S_t = \varnothing$. Given an audit point set S_t, the unchecked audit context reaching v is

$$\mathrm{Ctx}(v \mid S_t) = \{u \in \mathrm{Anc}_t(v) \cup \{v\} : u \rightsquigarrow_{S_t} v\}. \tag{8}$$

Equivalently, it is obtained by traversing backward from v and stopping each branch at the first selected audit point. This definition preserves unresolved branches in a DAG: if any path from u to v bypasses the selected audit points, u remains in the audit context. Thus, an audit point shortens only the downstream contexts whose information flow passes through it.

All audit points are selected before execution and triggered in causal order. Once an audit point passes the audit, its output becomes trusted input for downstream execution; an unsafe region is blocked and recovered before execution continues.

The cumulative exposure of v and the total exposure are

$$\begin{aligned} C(v \mid S_t) &= \sum_{u \in \mathrm{Ctx}(v \mid S_t)} w_t(u), \\ J(S_t) &= \sum_{v \in V_t} C(v \mid S_t). \end{aligned} \tag{9}$$

A high-exposure node can repeatedly contribute to downstream contexts until bounded by an audit point, increasing the audit burden and the cost of delayed detection. By summing over all nodes, $J(S_t)$ captures both the size of local audit contexts and the repeated inheritance of unchecked exposure by downstream computation.

Directly minimizing $J(S_t)$ is combinatorial, and audit point decisions are coupled because each selected audit point changes downstream contexts and the value of later selections. BRA-Audit therefore greedily selects audit points by marginal gain.

For a candidate $v \in V_t \setminus S_t$, let $\mathrm{Cut}_v(z \mid S_t)$ denote the exposure removed from the context of z:

$$\mathrm{Cut}_v(z \mid S_t) = \mathrm{Ctx}(z \mid S_t) \setminus \mathrm{Ctx}(z \mid S_t \cup \{v\}). \tag{10}$$

structure.

Let $\mathrm{Anc}_t(v)$ denote the ancestors of v. We write $u \rightsquigarrow_{S_t} v$ if there exists a directed path $p : u \rightsquigarrow v$ such that $(V(p) \setminus \{v\}) \cap S_t = \varnothing$. Given an audit point set S_t, the unchecked audit context reaching v is

$$\mathrm{Ctx}(v \mid S_t) = \{u \in \mathrm{Anc}_t(v) \cup \{v\} : u \rightsquigarrow_{S_t} v\}. \quad (8)$$

Equivalently, it is obtained by traversing backward from v and stopping each branch at the first selected audit point. This definition preserves unresolved branches in a DAG: if any path from u to v bypasses the selected audit points, u remains in the audit context. Thus, an audit point shortens only the downstream contexts whose information flow passes through it.

v: particular node we are examining

S_t: set of nodes selected for auditing at time t

p: directed path from u to v

V(p): set of all nodes along that path

$$\mathrm{Cut}_v(z \mid S_t) = \mathrm{Ctx}(z \mid S_t) \setminus \mathrm{Ctx}(z \mid S_t \cup \{v\}).$$

If we add an audit checkpoint at v, which nodes disappear from another node z's unchecked context?

1. For each node z, find which messages disappear from its unchecked context when we add checkpoint v. That is $\mathrm{Cut}_v(z|S_t)$.
2. Add the weights of those messages.
3. Sum these savings across all nodes z.

Table 1: Performance and token cost on AgentsNet. Token consumption is reported in thousands (K).

	Consensus		Leader		Matching		Coloring		VertexCover		Overall	
Method	**Score**	**Tok.**	**Score**	**Tok.**	**Score**	**Tok.**	**Score**	**Tok.**	**Score**	**Tok.**	**Score**	**Tok.**
Clean	0.9630	28.1	0.5926	29.9	0.6944	25.7	0.9967	31.1	0.8342	18.6	0.8162	26.7
Attack	0.0741	42.2	0.1481	43.1	0.4815	35.7	0.8324	47.5	0.6676	29.0	0.4407	39.5
PeerGuard	0.9630	69.3	0.5556	67.5	0.8056	68.1	0.9716	70.4	0.8852	53.6	0.8362	65.8
AuditAgent	0.8148	48.6	0.4444	50.2	0.6250	49.1	0.9013	53.9	0.7854	40.3	0.7142	48.4
GuardAgent	0.4074	44.7	0.3704	46.5	0.5370	39.3	0.8548	50.2	0.7014	33.4	0.5742	42.8
TrinityGuard	0.8889	58.0	0.4815	55.6	0.7385	51.8	0.8892	64.8	0.8156	44.9	0.7627	55.0
Multi-Agent Debate	1.0000	75.4	0.6296	74.5	0.8342	77.3	0.9967	81.6	0.8675	71.5	0.8656	76.1
BRA-Audit	0.9630	57.6	0.5185	57.0	0.7917	52.4	0.9967	61.7	0.8342	43.8	0.8208	54.5

The marginal gain of v is

$$\Delta(v \mid S_t) = J(S_t) - J(S_t \cup \{v\}) = \sum_{z \in V_t} \sum_{u \in \mathrm{Cut}_v(z \mid S_t)} w_t(u). \tag{11}$$

This quantity measures the exposure removed from all dependency contexts by adding v. It is large when v separates a high-exposure region from many downstream contexts. Since marginal gains are recomputed after each selection, the scheduler accounts for overlapping effects among audit points rather than scoring them independently. For a chain, the expression reduces to the accumulated prefix exposure multiplied by the number of affected downstream contexts, as illustrated in the technical appendix.

At each step, BRA-Audit selects

$$v^\star = \arg\max_{v \in V_t \setminus S_t} \Delta(v \mid S_t), \tag{12}$$

adds $v^\star$ to S_t, and updates the affected contexts until the audit budget is exhausted.

Online audit-point Placement and Audit Execution

BRA-Audit operates in an online manner. At the end of each MAS interaction round t, the system updates the dependency graph $G_t = (V_t, E_t)$. Before the next round starts, the scheduler uses this graph to place a limited number of audit points. Let $Q_{t+1} \subseteq V_t$ denote the candidate nodes for the next audit round. Given an audit ratio $\rho \in (0, 1]$, which corresponds to auditing $k\%$ of the candidates, the audit budget and audit point set are defined as

$$B_{t+1} = \lfloor \rho |Q_{t+1}| \rfloor, \qquad S_{t+1} \approx \arg\min_{S \subseteq Q_{t+1},\, |S| \le B_{t+1}} J(S). \tag{13}$$

Here, $J(S)$ is the cumulative unaudited exposure objective defined above. In practice, BRA-Audit selects audit points that provide the largest reduction of $J(S)$ until the budget is used. In the final round, the agent producing the final result is mandatorily selected and counted toward the budget.

After the audit-point set S_{t+1} is selected, each audit point is assigned to an auditor. For an audit point $s \in S_{t+1}$, the auditor inspects the audit-point bounded dependency region accumulated along all incoming branches:

$$\mathcal{T}(s \mid S_{t+1}) = \mathrm{Region}(s \mid S_{t+1}). \tag{14}$$

Here, $\mathrm{Region}(s \mid S_{t+1})$ is obtained by traversing backward from s and stopping each branch at the first upstream audit point in S_{t+1}. The audit point outputs are treated as trusted inputs, while the subsequent inputs, outputs, messages, and dependency steps form the local audit region.

If the auditor judges $\mathcal{T}(s \mid S_{t+1})$ as safe, s becomes a trusted audit point; otherwise, the system rolls back the work products affected by this region. By partitioning execution into audit-point bounded regions, BRA-Audit shortens audit contexts and localizes rollback. Because the auditor examines the entire region rather than only its endpoint, an unsafe agent can still be detected even when it is not selected as an audit point, provided that its inputs or outputs appear in the region of an downstream audit point.

Experiment

We evaluate BRA-Audit against recent LLM-agent auditing and multi-agent defense methods, focusing on defense effectiveness, audit cost, and robustness. Our experiments are designed to answer three research questions:

RQ1: Defense-Cost Trade-off. Can BRA-Audit achieve effective auditing while keeping the audit overhead low?

RQ2: Robustness Across Settings. How stable is BRA-Audit under different graph topologies, node importance distributions, attack locations, and backbone auditor models?

RQ3: Budget and Component Analysis. How do audit budget, audit point placement strategy, and exposure-weight components affect the overall performance of BRA-Audit?

Experimental Setup.

Dataset. We evaluate BRA-Audit on three complementary settings. First, we use AgentsNet (Grötschla et al. 2025), a multi-agent coordination benchmark with five graph-theoretic task and configurable communication topologies and node counts. This setting tests auditing under controlled multi-round agent communication. Second, we use BIG-Bench Hard (BBH) (Suzgun et al. 2022) to evaluate complex logical reasoning; we randomly sample 150 questions and convert each instance into a multi-agent reasoning workflow. Third, we use the Research scenario from MultiAgentBench (Zhu et al. 2025), an open-ended multi-agent task where agents collaborate to generate research ideas.

If v is selected, $Ctx(y \mid \{v\}) = \{y\}$

If x is selected, $Ctx\{y \mid \{x\}\} \subset \{a, b, c, v, y\}$

Table 2: Performance and token cost on BBH and Multi-agent-bench-rasearch.

	BBH			MultiAgentBench-Research					
Method	**Acc**	**recovery**	**token**	**innovation**	**safety**	**feasibility**	**Score**	**recovery**	**token(K)**
clean	0.9200	–	5355	5.00	4.62	3.83	0.897	–	23.0
attack	0.3933	–	6778	2.28	1.88	1.50	0.377	–	29.1
PeerGuard	0.8867	0.4933	10136	4.96	4.80	3.86	0.908	0.531	47.3
Auditagent	0.8067	0.4133	7459	3.88	3.92	3.14	0.730	0.352	32.4
Guardagent	0.5200	0.1267	7318	3.20	2.74	2.88	0.588	0.211	30.8
Trinityguard	0.8600	0.4667	8324	4.32	4.16	3.64	0.808	0.431	37.9
Multi-Agent Debate	0.9333	0.54	13763	4.98	4.86	4.02	0.925	0.548	60.5
BRA	0.8600	0.4667	8175	5.00	4.74	3.78	0.902	0.525	38.5

Table 3: Backbone model ablation on AgentsNet. Token consumption is reported in thousands (K).

	Consensus		Leader		Matching		Coloring		VertexCover		Overall	
Model	**Score**	**Tok.**	**Score**	**Tok.**	**Score**	**Tok.**	**Score**	**Tok.**	**Score**	**Tok.**	**Score**	**Tok.**
Qwen	0.9630	57.6	0.5185	57.0	0.7917	52.4	0.9967	61.7	0.8342	43.8	0.8208	54.5
DeepSeekV4-Flash	0.9630	51.0	0.5556	72.9	0.6435	64.2	0.9463	51.3	0.8250	44.6	0.7867	56.8
MiniMax-2.7	0.9259	55.8	0.5185	63.4	0.7592	51.3	0.9177	55.4	0.7935	35.6	0.7830	52.3
GLM-5.1	0.9259	62.9	0.5926	63.2	0.7314	56.8	0.9446	61.0	0.8250	53.1	0.8039	59.4
GPT-5	0.9630	78.5	0.6296	79.8	0.8333	81.7	0.9534	75.9	0.8956	65.1	0.8550	76.2
Gemini3-Flash	0.8889	48.7	0.5556	51.5	0.7963	50.1	0.9265	53.7	0.8342	40.2	0.8003	48.8

Settings. We implement all multi-agent systems with AutoGen (Wu et al. 2023), and instantiate agent networks with different sizes and communication topologies. In each setting, 20% of the agents is designated as malicious and is instructed to inject incorrect claims, misleading evidence, or corrupted intermediate results into the collaboration process to make the task fail **(detailed attack recovery protocol is provided in technical supplement)**. We mainly use Qwen3-235B-A22B-Instruct-2507 model. We further evaluate different backbone models, graph topologies, numbers of agents, and audit budgets ρ. The main metrics are task score and total E2E token consumption, which can measure defense effectiveness and cost. Each configuration is repeated three times, with the standard deviation kept below 2%.

Main Results.

We compare BRA-Audit with five representative LLM-based methods for runtime auditing or reliability enhancement. PeerGuard performs peer-based inspection during agent interactions (Fan and Li 2025). AuditAgent is a round-level auditing baseline adapted from AgentAuditor (Yang et al. 2026). GuardAgent converts scenario-specific guard requirements into executable checks (Xiang et al. 2024). TrinityGuard uses preassigned runtime monitors to inspect MAS execution (Wang et al. 2026). Multi-Agent Debate improves output reliability through iterative discussion among agents (Du et al. 2024). All methods use the same attack settings and backbone models.

The attack results confirm that LLM-MAS is vulnerable to a small number of corrupted agents. Even one or two malicious or hallucinating agents can propagate faulty outputs through communication and aggregation, substantially degrading task performance. On AgentsNet, the overall score falls from 0.8162 in the clean setting to 0.4407 under attack, a relative drop of approximately 46.0%. This supports auditing MAS collaboration is necessary.

On AgentsNet (Table 1), BRA-Audit restores the overall score to 0.8208 with 54.5K tokens. It outperforms AuditAgent, GuardAgent, and TrinityGuard, and matches the clean results on Consensus, Coloring, and VertexCover. Although PeerGuard and Multi-Agent Debate achieve slightly higher scores, BRA-Audit uses 17.2% and 28.4% fewer tokens, respectively. A similar defense–cost trade-off appears on BBH and MultiAgentBench-Research (Table 2). BRA-Audit improves BBH accuracy from 0.3933 to 0.8600, using 19.3% fewer tokens than PeerGuard and 40.6% fewer than Multi-Agent Debate. On Research, it reaches 0.902 with 38.5K tokens, close to both baselines while reducing token usage by 18.5% and 36.3%, respectively. Overall, BRA-Audit balances defense effectiveness and runtime cost across structured and open-ended tasks.

Ablation Study

Backbone Model Ablation. We evaluate BRA-Audit with six backbone models on AgentsNet benchmark to study its sensitivity to model capability(as shown in Table 3.) Overall scores range from 0.7830 to 0.8550, indicating stable behavior across all tested models. GPT-5 obtains the highest score of 0.8550, but also incurs the largest token cost at 76.2K. Qwen offers a better performance–cost balance, reaching 0.8208 with 54.5K tokens. Gemini-3 uses the fewest tokens (48.8K) while maintaining a score of 0.8003. GLM-5.1, DeepSeek-V4, and MiniMax-2.7 achieve 0.8039, 0.7867, and 0.7830, respectively. Performance differences are most visible on Leader and Matching. Overall, BRA-Audit remains effective across different backbones, while stronger models tend to improve task performance at higher token cost.

Table 4: Agent-count and topology ablations. Each entry reports Score / Avg. Token (K).

(a) Number of Agents

#Agents	Clean	Attack	BRA-Audit
4	0.859 / 8.3	0.363 / 14.1	**0.837 / 18.9**
8	0.827 / 21.3	0.520 / 33.4	**0.847 / 41.9**
16	0.763 / 50.4	0.439 / 71.0	**0.778 / 102.8**
Overall	0.816 / 26.7	0.441 / 39.5	**0.821 / 54.5**

(b) Graph Topology

Topology	Clean	Attack	BRA-Audit
WS	0.803 / 26.2	0.451 / 38.5	**0.846 / 55.6**
BA	0.835 / 25.2	0.415 / 38.1	**0.813 / 47.8**
DT	0.811 / 28.7	0.456 / 42.0	**0.804 / 60.1**
Overall	0.816 / 26.7	0.441 / 39.5	**0.821 / 54.5**

Table 5: Audit-budget ablation with audit and rollback costs. All token costs are reported in thousands (K).

	BBH				Research			
ρ	Score	Audit	Rollback	Total	Score	Audit	Rollback	Total
0.2	0.760	0.74	2.84	6.90	0.757	2.24	13.14	33.2
0.4, Rand.	0.820	2.14	1.76	8.01	0.821	5.43	9.13	37.4
0.4, **BRA**	**0.860**	2.09	1.81	8.18	**0.902**	5.82	9.38	38.5
0.6	0.873	3.91	1.35	9.57	0.906	9.11	7.17	42.9
1.0	0.893	5.57	0.94	11.31	0.897	11.13	5.64	46.8

Table 6: False-positive results under different audit ratios.

	BBH		Research	
ρ	FP	FPR	FP	FPR
0.2	0	0.00%	0	0.00%
0.4	7	4.67%	6	6.00%
1.0	18	12.00%	16	16.00%

Audit-Budget Ablation. We vary the audit ratio ρ on BBH and MultiAgentBench-Research to study the defense–cost trade-off. On BBH, increasing ρ from 0.2 to 0.4 improves the score from 0.760 to 0.860, while total token consumption rises from 6.9 to 8.18. Larger budgets of 0.6 and 1.0 further increase the score to 0.873 and 0.893, but require 9.57 and 11.31K tokens. MultiAgentBench-Research shows a similar trend: $\rho = 0.4$ achieves 0.902 with 38.5K tokens, close to the best score of 0.906 at $\rho = 0.6$. As ρ increases, audit cost grows while rollback cost decreases, indicating that more frequent auditing detects failures earlier and reduces re-execution. Under the same budget $\rho = 0.4$, BRA-Audit outperforms random audit-point placement by 0.040 on BBH and 0.081 on Research with comparable token costs. Overall, $\rho = 0.4$ provides a favorable balance among audit effectiveness, audit cost, and rollback overhead.

Node-Count Ablation. We evaluate BRA-Audit on AgentsNet graphs with 4, 8, and 16 agents (Table 4a). It achieves scores of 0.8370, 0.8473, and 0.7781, improving over the attacked setting by 0.4737, 0.3274, and 0.3391, respectively. Token consumption rises from 18.9K to 41.9K and 102.8K as larger graphs introduce longer communication paths and audit contexts. BRA-Audit therefore remains effective as the system scales, although larger networks require substantially more auditing resources.

Topology Ablation. We evaluate Watts–Strogatz, Barabási–Albert, and Delaunay graphs, which represent clustered small-world, hub-dominated, and spatially local communication structures, respectively (Grötschla et al. 2025). As shown in Table 4b, BRA-Audit obtains scores of 0.8458, 0.8127, and 0.8040, compared with attacked scores of 0.4511, 0.4153, and 0.4558. The small-world graph achieves the best score, while the preferential-attachment graph uses the fewest tokens. Overall, BRA-Audit remains effective across diverse communication topologies.

False-Positive Analysis. We assess false positives on clean BBH and MultiAgentBench-Research at varying audit ratios (Table 6). No FP occur at $\rho = 0.2$. At $\rho = 0.4$, FP rates are 4.67%/6.00% for BBH/Research, rising to 12.00%/16.00% under full auditing. Thus, excessive auditing can introduce unnecessary errors on benign trajectories. Most executions contain neither attacks nor severe hallucinations, so auditing every interaction incurs substantial redundant cost. Alongside task performance, BRA-Audit at $\rho = 0.4$ empirically lies on a favorable Pareto frontier balancing defense effectiveness, FP risk, and token consumption.

Weight-Component Ablation. We further ablate the two components of the exposure weight. Removing $\mathrm{AuditAge}$ decreases the overall score from 0.821 to 0.758, while removing $\mathrm{ClosedImpact}$ reduces it to 0.793. Both variants also increase total token consumption, showing that verification staleness and closed downstream influence jointly improve audit-point selection. Detailed results and analysis are provided in the **technical supplement**.

Conclusion

In this paper, we presented BRA-Audit, a cost-aware runtime defense for LLM MAS. It models MAS communication as a dependency graph and formulates auditing as cumulative-exposure audit point placement. By combining audit gaps with topological impact, BRA-Audit selects limited audit points in long-unaudited and influential regions, shortens audit contexts, limits unchecked propagation, and localizes rollback. Experimental results on structured coordination, complex reasoning, and open-ended tasks support a favorable balance between task recovery and token cost. demonstrates a favorable Pareto-optimal trade-off between task performance and auditing cost. We believe BRA-Audit can contribute to the development of more trustworthy and cost-aware MAS.

References

Cemri, M.; Pan, M. Z.; Yang, S.; Agrawal, L. A.; Chopra, B.; Tiwari, R.; Keutzer, K.; Parameswaran, A.; Klein, D.; Ramchandran, K.; Zaharia, M.; Gonzalez, J. E.; and Stoica, I. 2025. Why Do Multi-Agent LLM Systems Fail? In *Advances in Neural Information Processing Systems*.

Chan, C.-M.; Chen, W.; Su, Y.; Yu, J.; Xue, W.; Zhang, S.; Fu, J.; and Liu, Z. 2024. ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. In *The Twelfth International Conference on Learning Representations*.

Du, Y.; Li, S.; Torralba, A.; Tenenbaum, J. B.; and Mordatch, I. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 11733–11763. PMLR.

Fan, F.; and Li, X. 2025. PeerGuard: Defending Multi-Agent Systems Against Backdoor Attacks Through Mutual Reasoning. *arXiv preprint arXiv:2505.11642*.

Fourney, A.; Bansal, G.; Mozannar, H.; Tan, C.; Salinas, E.; Erkang; Zhu; Niedtner, F.; Proebsting, G.; Bassman, G.; Gerrits, J.; Alber, J.; Chang, P.; Loynd, R.; West, R.; Dibia, V.; Awadallah, A.; Kamar, E.; Hosn, R.; and Amershi, S. 2024. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. arXiv:2411.04468.

Grötschla, F.; Müller, L.; Tönshoff, J.; Galkin, M.; and Perozzi, B. 2025. AgentsNet: Coordination and Collaborative Reasoning in Multi-Agent LLMs. arXiv:2507.08616.

Gu, J.; Jiang, X.; Shi, Z.; Tan, H.; Zhai, X.; Xu, C.; Li, W.; Shen, Y.; Ma, S.; Liu, H.; Wang, S.; Zhang, K.; Wang, Y.; Gao, W.; Ni, L.; and Guo, J. 2025. A Survey on LLM-as-a-Judge. arXiv:2411.15594.

Guo, T.; Chen, X.; Wang, Y.; Chang, R.; Pei, S.; Chawla, N. V.; Wiest, O.; and Zhang, X. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*.

He, P.; Lin, Y.; Dong, S.; Xu, H.; Xing, Y.; and Liu, H. 2025. Red-Teaming LLM Multi-Agent Systems via Communication Attacks. In *Findings of the Association for Computational Linguistics: ACL 2025*, 6726–6747. Vienna, Austria: Association for Computational Linguistics.

Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Zhang, C.; Wang, J.; Wang, Z.; Yau, S. K. S.; Lin, Z.; Zhou, L.; Ran, C.; Xiao, L.; Wu, C.; and Schmidhuber, J. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*.

Ju, T.; Wang, Y.; Ma, X.; Cheng, P.; Zhao, H.; Wang, Y.; Liu, L.; Xie, J.; Zhang, Z.; and Liu, G. 2024. Flooding Spread of Manipulated Knowledge in LLM-Based Multi-Agent Communities. *arXiv preprint arXiv:2407.07791*.

Lee, D.; and Tiwari, M. 2024. Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems. *arXiv preprint arXiv:2410.07283*.

Li, G.; Hammoud, H. A. A. K.; Itani, H.; Khizbullin, D.; and Ghanem, B. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, volume 36, 51991–52008.

Li, X.; Wang, S.; Zeng, S.; Wu, Y.; and Yang, Y. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(1): 9.

Liu, N. F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; and Liang, P. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12: 157–173.

Miao, R.; Liu, Y.; Wang, Y.; Shen, X.; Tan, Y.; Dai, Y.; Pan, S.; and Wang, X. 2026. BlindGuard: Safeguarding LLM-based Multi-Agent Systems under Unknown Attacks. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 39215–39234. San Diego, California, United States: Association for Computational Linguistics.

Qian, C.; Liu, W.; Liu, H.; Chen, N.; Dang, Y.; Li, J.; Yang, C.; Chen, W.; Su, Y.; Cong, X.; Xu, J.; Li, D.; Liu, Z.; and Sun, M. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 15174–15186. Bangkok, Thailand: Association for Computational Linguistics.

Renze, M.; and Guven, E. 2024. Self-Reflection in LLM Agents: Effects on Problem-Solving Performance. *arXiv preprint arXiv:2405.06682*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.

Suzgun, M.; Scales, N.; Schärli, N.; Gehrmann, S.; Tay, Y.; Chung, H. W.; Chowdhery, A.; Le, Q. V.; Chi, E. H.; Zhou, D.; and Wei, J. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. arXiv:2210.09261.

Tran, K.-T.; Dao, D.; Nguyen, M.-D.; Pham, Q.-V.; O'Sullivan, B.; and Nguyen, H. D. 2025. Multi-Agent Collaboration Mechanisms: A Survey of LLMs. *arXiv preprint arXiv:2501.06322*.

Triedman, H.; Jha, R.; and Shmatikov, V. 2025. Multi-Agent Systems Execute Arbitrary Malicious Code. *arXiv preprint arXiv:2503.12188*.

Wang, J.; Wang, J.; Athiwaratkun, B.; Zhang, C.; and Zou, J. 2024. Mixture-of-Agents Enhances Large Language Model Capabilities. arXiv:2406.04692.

Wang, K.; Zeng, B.; Wei, Z.; Jin, C.; Zhou, H.; Li, X.; Yang, C.; Qu, J.; Xu, X.; and Hu, X. 2026. TrinityGuard: A Unified Framework for Safeguarding Multi-Agent Systems. *arXiv preprint arXiv:2603.15408*.

Wang, L.; Wang, W.; Wang, S.; Li, Z.; Ji, Z.; Lyu, Z.; Wu, D.; and Cheung, S.-C. 2025a. IP Leakage Attacks Targeting LLM-Based Multi-Agent Systems. *arXiv preprint arXiv:2505.12442*.

Wang, S.; Zhang, G.; Yu, M.; Wan, G.; Meng, F.; Guo, C.; Wang, K.; and Wang, Y. 2025b. G-Safeguard: A Topology-Guided Security Lens and Treatment on LLM-based Multi-agent Systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7261–7276. Vienna, Austria: Association for Computational Linguistics.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*.

Xia, Y.; Gao, A.; Quan, Y.; Liu, Z.; and Fang, M. 2026. Who Broke the System? Failure Localization in LLM-Based Multi-Agent Systems. *arXiv preprint arXiv:2607.07989*. To appear in the Conference on Language Modeling.

Xiang, Z.; Zheng, L.; Li, Y.; Hong, J.; Li, Q.; Xie, H.; Zhang, J.; Xiong, Z.; Xie, C.; Yang, C.; Song, D.; and Li, B. 2024. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. *arXiv preprint arXiv:2406.09187*.

Xie, Y.; Zhu, C.; Zhang, X.; Zhu, T.; Ye, D.; Wang, M.; and Liu, C. 2025. Who's the Mole? Modeling and Detecting Intention-Hiding Malicious Agents in LLM-Based Multi-Agent Systems. *arXiv preprint arXiv:2507.04724*.

Yan, B.; Zhou, Z.; Zhang, L.; Zhang, L.; Zhou, Z.; Miao, D.; Li, Z.; Li, C.; and Zhang, X. 2026. Beyond Self-Talk: A Communication-Centric Survey of LLM-Based Multi-Agent Systems. *Frontiers of Computer Science*.

Yang, W.; Li, S.; Ping, H.; Zhang, P.; Bogdan, P.; and Thomason, J. 2026. Auditing Multi-Agent LLM Reasoning Trees Outperforms Majority Vote and LLM-as-Judge. *arXiv preprint arXiv:2602.09341*.

Zhang, G.; Yue, Y.; Li, Z.; Yun, S.; Wan, G.; Wang, K.; Cheng, D.; Yu, J. X.; and Chen, T. 2024. Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. arXiv:2410.02506.

Zheng, C.; Cao, Y.; Dong, X.; and He, T. 2025. Demonstrations of Integrity Attacks in Multi-Agent Systems. *arXiv preprint arXiv:2506.04572*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685.

Zhu, K.; Du, H.; Hong, Z.; Yang, X.; Guo, S.; Wang, Z.; Wang, Z.; Qian, C.; Tang, X.; Ji, H.; and You, J. 2025. MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents. arXiv:2503.01935.